UNIDAD 1

Fundamentos de Machine Learning.

TEMA 2:

Principios de modelado y validación.

Descripción:

La asignatura Machine Learning 1 ofrece al estudiante una primera aproximación al fascinante mundo del aprendizaje automático, integrando teoría y práctica para resolver problemas reales con datos.

A lo largo del curso, se explorarán los fundamentos matemáticos y computacionales que sustentan los algoritmos modernos, junto con ejemplos aplicados en ciencia e ingeniería.

El estudiante aprenderá a diseñar, entrenar y evaluar modelos, comprendiendo tanto sus alcances como sus limitaciones.

Esta experiencia no solo fortalece competencias técnicas, sino también el pensamiento crítico, preparando a los futuros profesionales para enfrentar los desafíos de la era digital con creatividad e innovación.

Machine Learning 1

PERIODO ACADÉMICO 2026-1S

TABLA DE CONTENIDO

Tema 2. Principios de modelado y validación. 2
2.1. Flujo de un proyecto de aprendizaje automático. 2
2.2. Métricas de desempeño: accuracy, precision, recall, F1-score, RMS. 4
2.3. Validación cruzada y división de datos. 7
2.4. Sobreajuste y subajuste. 9
Bibliografía 13

Tema 2. Principios de modelado y validación.

Modelar y validar **no es** únicamente entrenar un algoritmo: es conectar una necesidad del dominio con datos, criterios de éxito, evaluación rigurosa y, finalmente, despliegue y monitoreo. Los estándares más recientes recomiendan trazabilidad, verificación continua y control de riesgos a lo largo del ciclo de vida del modelo. ISO/IEC 23053:2022 enmarca sistemas de IA basados en ML y NIST AI RMF 1.0 organiza la gestión del riesgo en funciones **GOVERN–MAP–MEASURE–MANAGE**, con énfasis en validez, confiabilidad y documentación de decisiones. Estas guías respaldan prácticas técnicas como separar conjuntos de datos, definir métricas primarias y secundarias, y evaluar sistemáticamente el desempeño antes y después del despliegue (ISO/IEC, 2022; NIST, 2023).

2.1. Flujo de un proyecto de aprendizaje automático.

El Flujo de un proyecto de aprendizaje automático se resume en cuatro etapas importantes que se ilustran en la Figura 6.

Figura 1. Proceso de Desarrollo de Modelos ML

Fuente: Elaboración propia

Formulación del problema y criterios de éxito

Traducir el objetivo del dominio a una tarea de ML exige precisar **variables, restricciones y costos de error**. Debe acordarse una métrica primaria (p. ej., *F1* o *AUROC* en clasificación; *RMSE* o *MAE* en regresión) y, cuando aplique, requisitos de **equidad, seguridad y explicabilidad**. NIST AI RMF 1.0 aconseja identificar riesgos y supuestos desde el diseño, y mantener evidencia de decisiones y controles de validación a lo largo del ciclo de vida (NIST, 2023).

Datos: adquisición, gobernanza y preparación

Un **pipeline** reproducible debe cubrir calidad de datos (valores faltantes, atípicos), **desbalance de clases**, *feature stores* y versionado. Una amenaza persistente es la **fuga de información** (usar, directa o indirectamente, datos del conjunto de prueba/validación para entrenar o diseñar transformaciones). Su prevalencia y efecto sobre la reproducibilidad ha sido documentado recientemente, enfatizando controles explícitos para evitarla (Kapoor & Narayanan, 2023).

Modelado y *feature engineering*

Se comparan familias de modelos bajo protocolos de evaluación consistentes y **espacios de hiperparámetros** definidos. Al ampliar la búsqueda (más modelos o hiperparámetros) aumenta el **sesgo por selección**: el "ganador por azar" tiende a verse mejor que su desempeño real. Métodos recientes permiten estimar y corregir este sesgo, reforzando comparaciones honestas (McLatchie et al., 2024).

Evaluación y selección

La evaluación requiere **divisiones adecuadas** (estratificación cuando procede) y **validación cruzada**. Cuando se optimizan hiperparámetros, la **validación cruzada anidada** (bucle interno para *tuning* y externo para evaluación) ofrece estimaciones menos optimistas del rendimiento del **procedimiento completo** (modelo + búsqueda) (scikit-learn developers, n. d.).

2.2. Métricas de desempeño: accuracy, precision, recall, F1-score, RMS.

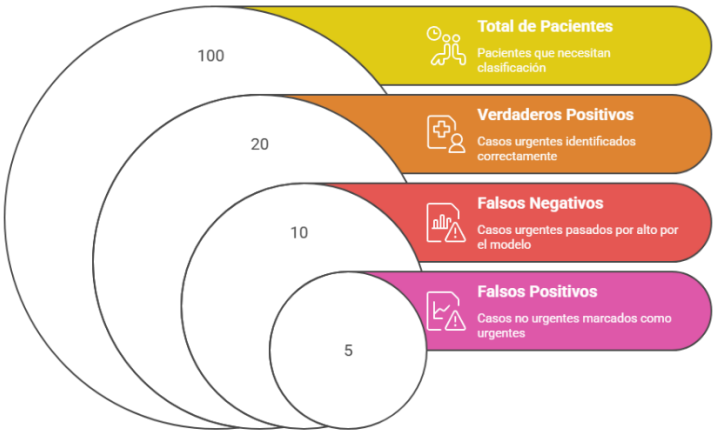
Para evaluar un modelo hay dos preguntas clave: **(i)** ¿distingue bien entre clases (o predice valores con poco error)?, y **(ii)** si entrega **probabilidades**, ¿éstas coinciden con lo que realmente ocurre?

A la primera la medimos con métricas **de discriminación** (accuracy, precision, recall, F1, AUROC/AUPRC; RMSE/MAE/R^2 en regresión); a la segunda, con métricas **de calibración** (Brier, log-loss, ECE). En contextos con **clase rara** (desbalance), conviene mirar métricas más informativas que la accuracy (Owusu-Adjei et al., 2023).

En un escenario de clasificación que se presenta en la Figura 7, imagina un modelo que decide si un paciente necesita **atención urgente** (positivo) o **no urgente** (negativo). En un lote de 100 pacientes, la realidad y el modelo producen:

- **VP** = 20 (urgentes bien detectados)
- **FP** = 5 (no urgentes marcados por error como urgentes)
- **VN** = 65 (no urgentes bien descartados)
- **FN** = 10 (urgentes que el modelo pasó por alto)

Figura 2. Resultados de Clasificación de Triage de Pacientes



Fuente: Elaboración propia

De esta manera, se describen las siguientes métricas de clasificación:

- $Accuracy = \frac{VP+VN}{TOTAL} = \frac{20+65}{100} = 0.85$

  Si los pacientes urgentes son pocos, la accuracy puede "verse bien" aunque el modelo ignore muchos casos críticos (Owusu-Adjei et al., 2023).

- $Precision = \frac{VP}{VP+FP} = \frac{20}{20+5} = 0.80$

  Esta métrica es útil cuando **cuesta** un falso positivo (activar una ambulancia innecesaria).

- $Recall = \frac{VP}{VP+FN} = \frac{20}{20+10} = 0.67$

  Clave cuando **no se puede perder** un urgente (minimizar falsos negativos).

- **F1** es la media armónica de Precision y Recall.

  Si **F1 sube**, equilibras mejor "acertar" y "no dejar pasar".

- **Balanced accuracy** es el promedio entre la sensibilidad + especificidad. Es muy útil cuando hay **desbalance** (pocos urgentes).

El modelo entrega una probabilidad $\hat{p}$. Si **subes el umbral** (p. ej., de 0.50 a 0.70): eres más estricto, entonces **sube** la precision, **baja** el recall (menos alarmas, pero se te escapan más urgentes).

Si **bajas el umbral** (p. ej., a 0.30): eres más permisivo, entonces **sube** el recall, **baja** la precisión (más alarmas, atrapas más urgentes). El umbral se fija por **costos** del dominio (NIST, 2023). Se prefiere recall ≥ 0.90 aunque baje la precisión, si la prioridad es no perder urgentes.

En un escenario de **regresión**, se desea predecir la **resistencia** (en MPa) de 5 probetas:

Valores verdaderos de resistencia: $y = [10, 12, 9, 11, 8]$

Valores predichos de resistencia: $\hat{y} = [9, 13, 8, 12, 10]$

De esta manera, se describen las siguientes métricas de regresión:

- El **error** punto a punto es

  $e = y - \hat{y} = [-1,1,-1,1,2]$

- **MAE (error absoluto medio)**

  $$MAE = \frac{|-1|+|1|+|-1|+|1|+|2|}{5} = 1.2\text{MPa}$$

En promedio, el modelo se equivoca **1.2 MPa**. Es una medida **robusta** frente a valores atípicos.

- **RMSE (raíz del error cuadrático medio)**

  $$RMSE = \sqrt{\frac{(-1)^2+(1)^2+(-1)^2+(1)^2+(2)^2}{5}} = 1.26\text{MPa}$$

Se penaliza **más** los errores grandes y es útil cuando los desvíos altos son costosos (Hodson, 2022).

- El **coeficiente de determinación R^2** mide la proporción de la variabilidad de la variable objetivo y que explica un modelo respecto de un modelo base que solo usa la media $\overline{y}$

2.3. Validación cruzada y división de datos.

Para estimar la generalización de un modelo, no basta con medirlo sobre los mismos datos con los que aprendió. Por eso se **divide** el conjunto en partes con roles distintos y se usa **validación cruzada (CV)** para obtener una evaluación más estable y realista.

La división básica de datos en **entrenamiento, validación y prueba** consiste en separar el conjunto total para asignar roles distintos: con el **entrenamiento** el modelo aprende sus parámetros; con la **validación** se eligen los hiperparámetros y se comparan configuraciones; y con la **prueba** se realiza, al final, un "examen" verdaderamente fuera de muestra para estimar la generalización. Cuando existe **desbalance de clases** (por ejemplo, solo 10 % de positivos), es fundamental **estratificar** los cortes para mantener la misma proporción de clases en cada partición y así evitar métricas engañosas. Un ejemplo breve en clasificación: si cuentas con 1000 casos y un 10% de positivos, puedes aplicar un **70/15/15** estratificado, quedando 700 instancias para entrenar, 150 para validar y 150 para probar; de este modo, cada subconjunto conserva aproximadamente ese 10% de positivos y las métricas obtenidas serán comparables y más confiables.

Figura 3. División del conjunto de datos para el desarrollo del modelo

Fuente: Elaboración propia

La **validación cruzada (K-fold)** reemplaza el uso de un único corte por varios pliegues para obtener una estimación más estable del desempeño. Consiste en dividir el conjunto en KKK partes (por ejemplo, K=5K=5K=5); en cada ronda se entrena con K−1K-1K−1 pliegues y se valida con el restante, repitiendo el proceso KKK veces y promediando las métricas resultantes. Así se reduce la dependencia del azar propia de una sola partición. A modo de ilustración con 10 muestras y K=5K=5K=5: en la **Ronda 1** se entrena con las muestras 1–8 y 10 y se valida con la 9; en la **Ronda 2**, se entrena con 1–7 y 9–10 y se valida con la 8; y así sucesivamente hasta completar las cinco rondas. Al final, el promedio de las cinco puntuaciones proporciona una única estimación de rendimiento.

Cuando además se buscan hiperparámetros óptimos —por ejemplo, la profundidad de un árbol— se recomienda **validación cruzada anidada**: un bucle interno para afinar (selección de hiperparámetros) y un bucle externo para evaluar el procedimiento completo. Este esquema evita el

optimismo de "probar y examinar" con los mismos pliegues y ofrece una medida más honesta de generalización.

En **series temporales** nunca debe mezclarse pasado y futuro. La división debe respetar la cronología: por ejemplo, entrenar con datos de enero a marzo y validar con abril; luego, entrenar con enero a abril y validar con mayo. Este patrón simula el entorno de producción y previene evaluaciones irreales por "mirar el futuro".

Por último, hay que vigilar las **fugas de información (data leakage)**, que aparecen cuando el modelo, sin querer, "ve" datos del conjunto de prueba o del futuro durante el preprocesamiento (como normalizar con todo el dataset o crear variables usando información futura). Para prevenirlas, se debe **dividir primero** y encapsular todo el preprocesamiento en un **pipeline** que se ajuste exclusivamente con el conjunto de entrenamiento dentro de cada pliegue. En síntesis, dividir bien (train/validation/test, con estratificación cuando corresponda) y evaluar con validación cruzada —preferentemente **anidada** si ajustas hiperparámetros y **respetando el tiempo** en series temporales— es la base para reportar resultados confiables, reproducibles y libres de inflaciones accidentales de las métricas.

2.4. Sobreajuste y subajuste.

El **sobreajuste** aparece cuando el modelo "memoriza" detalles o ruido del conjunto de entrenamiento y, por ello, pierde capacidad de **generalizar** a datos nuevos; el **subajuste**, en cambio, refleja que el modelo es demasiado simple o está mal especificado y no logra captar los patrones esenciales.

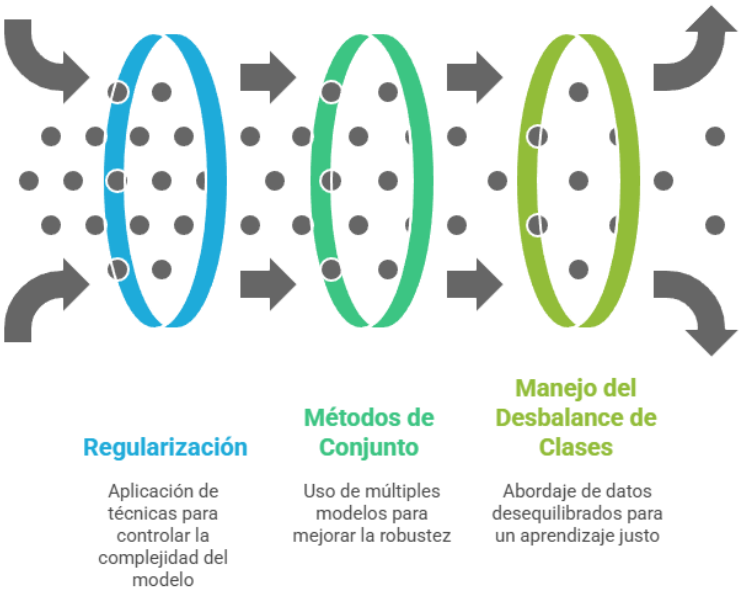
Una manera didáctica de distinguirlos es observar **curvas de aprendizaje** (error frente al tamaño del conjunto) y **curvas de validación** (error frente a un hiperparámetro de complejidad).

Si al añadir más datos el error de validación cae y el de entrenamiento sube ligeramente hasta estabilizarse, suele haber margen para mejorar generalización; si el error de entrenamiento ya es alto y el de validación también, es probable que exista subajuste; y si el error de entrenamiento es muy bajo pero el de validación permanece alto, el diagnóstico típico es sobreajuste y conviene regularizar o simplificar.

Frente al sobreajuste, la **regularización** controla la complejidad efectiva del modelo. La penalización **L2** (ridge) distribuye el peso entre los parámetros y reduce la varianza del estimador; la **L1** (lasso) además **induce sparsidad**, útil cuando muchas variables aportan poco. En redes neuronales, **dropout** actúa como una regularización estocástica que evita coadaptaciones entre neuronas. Un complemento práctico es la **parada temprana (early stopping)**, que consiste en monitorizar el error de validación durante el entrenamiento y detener el proceso cuando dicho error deja de mejorar de manera sostenida; así se evita que el optimizador "sobre-entrene" hasta capturar ruido del set de entrenamiento (Salehin et al., 2023). En términos operativos, combinar una penalización moderada con *early stopping* suele ofrecer un equilibrio sólido entre sesgo y varianza en tareas modernas.

Otra familia de técnicas que ayuda a mitigar sobreajuste y mejorar la **robustez** son los **conjuntos (ensembles)**. El **bagging** (por ejemplo, *random forests*) reduce la varianza al promediar muchos modelos entrenados sobre muestras re-muestreadas; el **boosting** (como *XGBoost* o *LightGBM*) corrige sistemáticamente el sesgo ajustando modelos débiles de forma secuencial sobre los errores residuales; y el **stacking** combina predictores heterogéneos mediante un modelo de nivel meta que aprende a ponderarlos.

Figura 4. Proceso de Optimización del modelo



Fuente: Elaboración propia

En la práctica, estos métodos tienden a estabilizar el rendimiento ante ruido y desbalance, y a capturar relaciones no lineales sin necesidad de un único modelo muy complejo (Mienye & Sun, 2022). Eso sí, el uso de *ensembles* no exime de un diseño de validación correcto: si hay fuga de información en el *pipeline*, cualquier mejora aparente será ilusoria.

El **desbalance de clases** exige decisiones explícitas porque agrava tanto el subajuste (la clase minoritaria "no se aprende") como el sobreajuste (el modelo memoriza pocos positivos). Tres palancas didácticas son: **ajustar el umbral de decisión** a los costos del dominio (por ejemplo, priorizar *recall* para reducir falsos negativos en salud), **ponderar clases** (p. ej., class_weight="balanced" para que el error pese más en la minoría) y aplicar **re-muestreo** estratégico (sobre-muestreo de la minoría con **SMOTE** o variantes, sub-muestreo de la mayoría o combinaciones). Sea cual sea la opción, el preprocesamiento debe ejecutarse **dentro de la validación cruzada** (no antes) para evitar fugas y sobreestimaciones del

rendimiento. Además, conviene reportar métricas sensibles al desbalance y, si se usan probabilidades, comprobar la **calibración**.

En conjunto, el itinerario práctico es: diagnosticar con **curvas de aprendizaje/validación** y métricas adecuadas; aplicar **regularización** y, de ser necesario, **parada temprana**; considerar **ensembles** para ganar robustez sin inflar la complejidad de un único modelo; y tratar el **desbalance** con umbrales, ponderación o re-muestreo, siempre bajo un esquema de evaluación reproducible y sin fugas. Esta combinación, respaldada por documentación técnica y revisiones recientes, es la base para modelos que generalizan mejor y métricas que reflejan su comportamiento real (Mienye & Sun, 2022; Salehin et al., 2023).

Bibliografía

Barthels, C., Müller, I., Taranov, K., Alonso, G., & Hoefler, T. (2019). Strong

Dritsas, E., Stamelos, I., Tsakalidis, A., & Sotiropoulos, D. N. (2025). Exploring the intersection of machine learning and big data. *Journal of Open Innovation: Technology, Market, and Complexity, 7*(1), 13. MDPI. https://doi.org/10.3390/make7010013

Hao, Z., Liu, S., Zhang, Y., Ying, C., Feng, Y., Su, H., & Zhu, J. (2022). Physics-informed machine learning: A survey on problems, methods and applications. *arXiv preprint*. https://arxiv.org/abs/2211.08064

Hodson, T. O. (2022). Root-mean-square error (RMSE) or mean absolute error (MAE): When to use them or not. *Geoscientific Model Development, 15*, 5481–5487. https://doi.org/10.5194/gmd-15-5481-2022

IBM. (2023). *Five machine learning types to know*. IBM Think. https://www.ibm.com/think/topics/machine-learning-types

ISO. (2023). *Artificial intelligence — Machine learning — Framework for machine learning*. International Organization for Standardization. https://www.iso.org/artificial-intelligence/machine-learning

ISO/IEC. (2022). ISO/IEC 23053:2022 — Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML). ISO. ScienceDirect

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in ML-based science. *Patterns, 4*(10), 100779. https://doi.org/10.1016/j.patter.2023.100804

McLatchie, Y., et al. (2024). Efficient estimation and correction of selection-induced bias. *Statistics and Computing, 34*, 132. https://doi.org/10.1007/s11222-024-10442-4

Mersha, M., Lam, K., Wood, J., Alshami, A. K., & Kalita, J. (2024). Explainable artificial intelligence: A survey of needs, techniques, applications, and future direction. *Neurocomputing*, *599*, 128111. https://doi.org/10.1016/j.neucom.2024.128111

Mienye, I. D., & Sun, Y. (2022). A Survey of Ensemble Learning: Concepts, Algorithms, Applications, and Prospects. *IEEE Access, 10*, 99129–99159. https://doi.org/10.1109/ACCESS.2022.3207287

AI, N. (2023). Artificial intelligence risk management framework (AI RMF 1.0). *URL: https://nvlpubs. nist. gov/nistpubs/ai/nist. ai*, 100-1

Owusu-Adjei, M., Ben Hayfron-Acquah, J., Frimpong, T., & Abdul-Salaam, G. (2023). Imbalanced class distribution and performance evaluation metrics. *PLOS Digital Health,* 2(11), e0000290. https://doi.org/10.1371/journal.pdig.0000290

scikit-learn developers. (n. d.). 3.1 Cross-validation: evaluating estimator performance. *Scikit-learn 1.7.2 documentation*. scikit-learn.org

scikit-learn developers. (n. d.). Nested versus non-nested cross-validation (example). *Scikit-learn 1.7.2 documentation*. scikit-learn.org

scikit-learn developers. (n. d.). Calibration tools (brier_score_loss, calibration_curve). *Scikit-learn 1.7.2 documentation*. scikit-learn.org

Symeonidis, G., Nerantzis, E., Kazakis, A., & Papakostas, G. A. (2022, January). Mlops-definitions, tools and challenges. In *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)* (pp. 0453-0460). IEEE. https://doi.org/10.1109/CCWC54503.2022.9720902

Yazici, İ., Yıldız, O. T., & Karakaya, M. (2023). A survey of applications of artificial intelligence and machine learning. *ICT Express,* 9(3), 221–233. https://doi.org/10.1016/j.jestch.2023.101455